ARCHITECTURE GUIDE

How the Bitcoin-backed USDC one-loan system works

The system coordinates one loan of USD Coin (USDC) on the Base network between one borrower and one selected lender, secured by bitcoin that the borrower locks in a Bitcoin escrow. It is built so that each asset moves only through fixed routes that the contract rules and the participants' own signatures allow (who controls upgrades to the Base contract is one of the open questions), and so that the product and the organization running it never hold lender capital, a collateral key or a participant's unrevealed secrets.

> This is the architecture guide of Bitcoin-backed USDC one-loan system. It explains how the system works as a whole. Each of its five domains has a separate domain guide: Loan Terms And Commitments, Bitcoin Authorization And Recovery, Base USDC Settlement And Funding, Base Observation And Fallback Review and Participant Access And Loan Operations.

In this guide

What the system is for 1
The five domains and their roles 3
A loan from offer to repayment 4
What binds everything to one loan 7
Who decides what, and what counts as proof 7
When things go wrong 8
What has been built and what is claimed 12
Choices and their reasons 12
Open questions 13

What the system is for

The product goal is simple to state: a borrower pledges bitcoin, receives USDC from a lender, repays, and gets the bitcoin back. The difficulty is that the loan lives on two chains that cannot see each other. Base records cannot spend or confirm Bitcoin, and Bitcoin cannot inspect Base repayment. The design keeps the two sides consistent through fixed, pre-authorized routes, and keeps authorities separate so that no term record, product identity, participant secret, observer, external provider or chain fact acquires another boundary's authority.

Who takes part

- The **borrower** stages bitcoin as collateral, accepts the funded principal, and repays the exact debt.

- The **selected lender** publishes a firm offer for one loan and later funds it with an explicit action from a wallet or treasury they control; the product cannot pool, hold, substitute or automatically deploy that capital.
- Each participant has a dedicated **Vault**: a separate browser application that owns the participant's vault password, seed, per-loan Bitcoin keys and role secrets, and does its decryption, review and signing inside its own browser origin. Signing in to the product only gives access to encrypted data; it is not decryption, signing, secret recovery or chain authority.
- The **organization** operates the product, the application service and the Bitcoin automation. It has no key in the collateral script. Its Bitcoin fee wallet may fund a child transaction that adds fee to an already signed transaction (child-pays-for-parent, CPFP, through a pay-to-anchor output, P2A) without changing that signed transaction, and its Base fee wallet may submit permissionless fixed-effect calls, without gaining recipient, capital, debt, secret or contract authority.
- An **evidence-conditioned validation service (ECVS)** is a selected design for an independent service that, for one narrow failure case described under When the lender stays silent after valid repayment, could publish a per-loan secret. Its production availability remains gated: counsel approval and implementation evidence must activate it before any loan may commit to it or present it as available.
- A **developer** runs the whole system locally; that local demonstration is the target outcome of the current design and is still in progress (see What has been built and what is claimed).

Concepts you need

- **Prefund and escrow.** The borrower first sends bitcoin to a prefund output. It becomes the loan's escrow only when an activation transaction spending it reaches six confirmations. Staged collateral, package preparation, Base acceptance and active collateral are kept as separate states and never collapse into one.
- **Packages.** Apart from the borrower's prefund, which she signs alone, and a cooperative settlement, which both participants sign freshly when they agree, each movement of the collateral uses a package: a transaction the participants signed before it is needed, stored as a partially signed Bitcoin transaction (PSBT) with a manifest. Each pays a fixed destination, so completing and broadcasting it later needs only the matching public condition, such as a revealed secret or a matured timelock. Because fees cannot be changed after signing, each package is signed in six fee variants.
- **Three secrets.** `X` is the borrower's acceptance secret, `Y` is the lender's claim secret, and `S` is the per-loan secret the ECVS could publish, only for a loan created after the fallback's production activation with the hash of `S` committed in its terms. Revealing `X` on Base accepts the loan and selects one fixed activation; revealing `Y` or `S` satisfies the same fixed borrower-recovery transaction. A public secret is fixed-route execution capability, not arbitrary signing, destination selection or chain authority.
- **Deadlines.** `D` is the repayment deadline. The lender's claim cutoff is `C = D + 172,800` seconds (48 hours). The lender's timelocked Bitcoin route becomes valid after `T = D + 259,200` seconds (72 hours). Bitcoin maturity is measured by consensus time (median time past, MTP),

never by an application or Base clock.

- **Events are not outcomes.** A command can be accepted, a job completed, a transaction submitted, a Base event observed or finalized, and a Bitcoin transaction broadcast or confirmed. These are different facts, and the system reports each separately rather than one generic completion.

The five domains and their roles

The system is divided into five domains. The separation matters because each can report progress without acquiring another's authority: login is not a Vault unlock, an accepted command is not a chain fact, and a Base refund is not a confirmed Bitcoin recovery.

Domain	Its role and boundary
Loan Terms And Commitments	Owns the canonical meaning and immutable identity of the one-loan offer, the borrower reservation, the final terms and the environment-wide deployment facts. The binding grants no signing, secret-disclosure, funding, payout-selection, chain-observation or execution authority.
Bitcoin Authorization And Recovery	Owns participant-controlled Bitcoin keys and the prefund, escrow, activation, refund, recovery, default, cooperative-settlement, fee, relay and observation mechanisms, without giving the product, Base, the ECVS or the organization collateral authority.
Base USDC Settlement And Funding	Owns the Base loan record and the immutable per-loan funding and repayment vaults: selected-lender funding, exact repayment, the lender's claim and fixed-recipient refunds, without becoming a Bitcoin executor or a custodian of lender capital.
Base Observation And Fallback Review	Decides which Base events count and when they are final, with provenance, correction and replay, and holds the conditional ECVS boundary. It excludes Base execution, Bitcoin signing, destination choice and discretionary evidence arbitration.
Participant Access And Loan Operations	Owns product accounts, roles and access, the durable loan workspace with its commands, jobs, projections and package archive, the hand-offs to each Vault (expiring pointers, not signing authority), and the isolated local demonstration. It coordinates already-authorized actions and owns no participant private authority, no lender capital and no independent Bitcoin or Base truth.

Every domain identifies the loan through the binding that Loan Terms And Commitments defines. Participant Access And Loan Operations carries each narrowly bound step to the participant's Vault and to the Base contracts. A hand-off to the Vault is an expiring pointer, not authentication, transaction consent or cryptographic authority, and the Vault never signs bytes supplied only by the product page. Base Observation And Fallback Review turns contract events into finalized facts that the workspace and the Bitcoin automation may rely on, and Bitcoin Authorization And Recovery takes a Base fact into account only once it is finalized, relying on its own independent observation for Bitcoin facts.

A loan from offer to repayment

The following case is illustrative: Ana borrows, Leo lends, and everything goes to plan. The exceptions follow in When things go wrong.

Agreeing the loan

Leo publishes a firm one-loan offer, which borrowers can browse. Ana selects it, and in one atomic step the system reserves the offer for her for 24 hours, in which she must produce a valid prefund, creates the stable loan identity and records the commercial draft; a concurrent attempt loses without creating a second reservation. Reservation does not fund USDC, move bitcoin or create signing authority. What must happen next before the terms are binding is an open question in the records. The architecture record says that publication is the lender's standing acceptance and that reservation creates the canonical loan without a second lender approval; the offer-and-reservation interface, the exclusion record for multi-lender matching and the offer selection also say there is no second lender approval. The loan-terms responsibility says that neither publication nor reservation approves or commits the final bound terms, and it, the terms interface and the origination flow have Ana and Leo each explicitly approve the same final candidate, with the second matching approval committing it exactly once. Neither side is settled until the user answers.

While the reservation holds, each Vault allocates a fresh per-loan Bitcoin key, confirms that role's payout destinations, and generates the role's secret if it does not exist yet: `X` for Ana, `Y` for Leo. Each Vault returns only public keys, addresses and secret hashes; signing in, choosing destinations or acknowledging these results is not secret disclosure, approval, decryption or signing authority. In the origination flow's ordering, which is one side of that undecided question (the other holds that reservation already created the loan without a second lender approval), the complete final candidate is assembled from these results, and committing it produces the immutable terms and their hash; approval grants no signing or custody authority, and a change to the terms invalidates earlier approvals.

Staging the collateral

Ana broadcasts the exact prefund. Once the terms are committed and the prefund is observed, even with zero confirmations, the Bitcoin domain builds six activation candidates and six refund candidates for it. Ana signs all twelve; Leo verifies the six activation candidates and adds his signature to them. The twelve signed transactions, the witness script, the manifests and their checksums are written as immutable records to the primary archive in Cloudflare, and the archive receipt is validated; an encrypted backup outside Cloudflare follows on a schedule, and a pending or failed backup raises an alert without blocking the loan. The attempt qualifies for funding only after six unchanged confirmations, fresh agreement between independent observers, and with at least 67 hours left before the borrower's prefund-refund route matures. A zero-confirmation prefund is not collateral, and qualification moves no USDC, reveals no secret and creates no escrow.

Funding and acceptance

A narrowly authorized opener opens the Base funding window and deploys an immutable funding vault for this loan. After his explicit funding approval, Leo transfers USDC to it from any wallet or treasury he controls, and anyone may then finalize the vault once it holds at least the principal; exactly the principal is taken. A transfer, a balance or a submitted transaction is only provisional: funding counts when the funding event reaches Base layer-1 (L1) finality on Ethereum, and the sender gains no loan role.

Ana then accepts in her Vault, in two confirmations. She first confirms a review of the nonsecret preparation; the Vault then minimally unlocks `X` and her Base key and freezes the exact acceptance transaction, and a second confirmation bound to those exact bytes lets the Vault sign and submit it. The first confirmation is not consent to sign, a failure or cancellation clears the plaintext, and the application never receives `X` or signs the call. Acceptance needs funding no older than 24 hours and at least 19 hours left before the prefund refund matures. On acceptance the contract atomically registers a separate repayment vault, pays the principal to Ana's committed recipient and makes `X` public. Base acceptance is not Bitcoin activation: from here until the expected escrow output has six confirmations, the loan is activation pending, and if one of Ana's prefund-refund transactions confirms first, Leo's principal is left unsecured (see When Base pays but Bitcoin activation does not confirm).

Activating the escrow

Once `X` is final on Base, the platform selects one activation candidate, inserts `X`, and broadcasts only that candidate. The other signed candidates remain valid competitors until one of them spends the prefund, and whichever actually reaches six confirmations becomes the escrow; if one of Ana's prefund-refund transactions confirms first instead, no escrow exists and Leo's principal is unsecured. Once an escrow has six confirmations, the Bitcoin domain builds the terminal packages for it: Ana's normal-recovery variants and Leo's lender-recovery variants, each paying its signer's fixed destination. Whether Ana's long-stop variants are also built and signed in this phase is open, because the records disagree whether it holds twelve or eighteen packages (see Open questions). Each participant signs only their own routes. Whether the long-stop protects a borrower after valid repayment is undecided: a build constraint requires every new escrow to include this route, while the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route (see When the lender stays silent after valid repayment). When these are stored and the archive receipt validates, the loan is package-ready active. Until then it stays explicitly terminal-signing-pending; that state keeps prompting the affected participant but cannot reverse the paid principal or the confirmed escrow, and it must not be presented as package-ready.

Repaying and closing

Before `D`, anyone may transfer the exact debt to the repayment vault, including from a wallet that is not Ana's; paying grants no loan role, refund right or destination control. Anyone may then finalize the repayment, which is valid only when the exact debt from the registered vault is final on Base no later than `D`. Leo claims strictly before `C`: after fresh reauthentication and multi-factor authentication, he first confirms a review of the nonsecret claim; his Vault then unlocks `Y`, and a second confirmation bound to the exact bytes lets it sign and submit the claim; a failure or

cancellation clears the plaintext. The contract pays the exact debt once to Leo's fixed recipient and makes Y public. Once Y is final on Base, the platform inserts it into Ana's already signed recovery transaction and broadcasts it, with no fresh signature and no changed output. The Bitcoin side is complete only when that spend confirms; the application does not report completion before confirmation.

The figure condenses the case. Each hand-off between the chains waits for a final fact on the other chain, and the Base and Bitcoin steps are never one atomic transaction; between acceptance and activation, Leo bears the risk that Ana's prefund refund confirms first:



Figure 1. Base acceptance and Bitcoin activation are separate steps; the borrower's bitcoin is back only when the recovery transaction confirms.

What binds everything to one loan

Every domain identifies the same loan through one canonical terms value, `LoanTermsV1`, and its terms hash. The binding covers the two participants, asset and amount terms, deadlines, the loan-specific Bitcoin public keys, independently selected external Bitcoin destinations, Base protocol addresses and fixed settlement recipients, and the commitments to `X`, `Y` and the conditional `S`. Whether it also carries the selected offer revision and reservation number is unresolved: the architecture record and the loan-terms responsibility say it does, while the terms interface says offer and reservation identity belongs to the offer-and-reservation contract, not the terms value. A separate deployment manifest, `ProtocolDeploymentV1`, binds environment-wide facts: chain, contracts, asset, Bitcoin network, timing policy and upgrade authority. Neither commitment is a wallet, a signing key, a source of funds, a chain-state fact or a lifecycle action. Any Base record, Bitcoin artifact, workspace record, observation or secret commitment that names the wrong loan, chain, asset, key, destination or version is rejected, and no workspace copy or acceptance snapshot can redefine committed terms.

A participant's Vault payment account is a separate public source-of-funds identity, distinct from the protocol account that makes the participant's Base calls, from every loan key and from every organization wallet. Selecting its address as a fresh loan's existing recipient value requires fresh approval and does not rewrite historical terms. The capability's delivery status is itself an open question: the payment selection records it as partial, with implementation and execution evidence still gated, and the local-demo browser verification records it as planned and unverified, while the core coordination hand-off says historical payment implementation is already recorded and the hand-off sequence records a correction that passes focused probes without finalized receipts. At most it is a selected design whose implementation and verification are unconfirmed.

Who decides what, and what counts as proof

Authority is split so that no record, identity, secret, observer, provider or chain fact acquires another boundary's authority:

- **Loan meaning.** Loan Terms And Commitments owns the canonical meaning, the offer and reservation identity, the terms versions and the binding.
- **Participant secrets and signatures.** Each participant's Vault owns the password, seed, loan keys and unrevealed `X` or `Y`, and signs the participant's own Base calls and Bitcoin packages. The product, the service, operators, logs and the package archive never receive them.
- **Capital.** The selected lender owns capital approval and funding. The product presents the immutable funding vault and may accept a participant's Vault payment account as one eligible payer, but it cannot pool, hold, substitute or automatically deploy lender capital. Whether that payment capability is only selected with its evidence gated or already implemented is undecided between its records.
- **Base state.** The Base contracts own funding, acceptance, repayment, claim, fixed-recipient refund, deadlines and the public `X` and `Y`. Raw USDC transfers, vault balances, submitted transactions and application status remain provisional until the named contract event reaches the approved finality boundary.

- **Bitcoin state.** Bitcoin consensus owns the prefund and escrow outputs, signatures, timelocks, broadcasts, confirmations, reorganizations and the one final spend of the escrow.
- **Observation.** The Base observer reports an authoritative fact with its provenance and finality; it does not create the fact or approve its consequence. Independent Bitcoin observers do the same for Bitcoin and never construct, relay, price, sign or authorize Bitcoin actions.
- **Coordination.** The loan workspace owns durable commands, the job outbox, sourced projections, alerts, package custody and the coordination of already-authorized actions, and owns no participant private authority, no lender capital and no independent Bitcoin or Base truth.
- **The conditional ECVS.** For a loan created after its production activation with the hash of `S` committed, it may verify the exact finalized Base refund event and publish only that loan's committed `S`. It is not an evidence arbiter, a Bitcoin signer, a destination selector or a discretionary release authority, and its production availability remains gated.
- **The local demonstration.** Its composition owns only the explicit local-demo profile and cannot redefine protocol meaning, become a chain source of truth or activate a public profile.

What counts as proof differs by chain. The application acts on a Base fact only after the block containing it reaches Base L1 finality; it may display an event earlier, labeled as awaiting finality, but creates no lifecycle transition, Bitcoin action or completion status from it. Base L1 finality is chosen as one consistent rule at every consequential cross-chain boundary, accepting extra waiting rather than promising an exact waiting time. On the Bitcoin side, a relay's response proves nothing; independent observation owns mempool and confirmation facts, and only a confirmed spend settles an outcome. When `X` or `Y` becomes final on Base, or `S` is validly published, the secret permits only the already-authorized matching Bitcoin route: Base facts and public secrets never sign, redirect, broadcast or confirm Bitcoin by themselves.

When things go wrong

When origination stops before acceptance

If funding or acceptance expires, or the attempt is cancelled or fails before `X` is final on Base, the two sides unwind separately. Anyone may permissionlessly return Leo's principal, and any excess or late USDC, only to his fixed lender recipient. Ana's bitcoin returns through one of her signed prefund-refund transactions once its relative timelock matures, the prefund is unchanged and unspent, two observers agree, and Base shows no accepted `X`; the application cannot satisfy that gate from a wall clock, a stale source or a relay response. There is no company-held `X`, delegated acceptance or company activation authority: when the borrower does not accept the funded principal, funding expires and the loan unwinds this way, and automatic activation recovery after borrower nonresponse is deferred. Once `X` is final, this unwind is forbidden and activation recovery continues instead.

When Base pays but Bitcoin activation does not confirm

Base acceptance and Bitcoin activation are not one atomic transaction. After `X` is final, recovery must keep pursuing the same selected activation; it cannot refund the prefund, pay principal again or construct an unapproved replacement. If activation is delayed long enough that the

borrower's prefund refund matures and confirms first, Ana keeps both the principal and her bitcoin, and Leo is left with unsecured credit exposure. This is the principal residual cross-chain risk, and it is the lender's to bear: no component may claim literal atomicity or mark the loan active before the expected escrow has six confirmations.

When the lender stays silent after valid repayment

Suppose Ana's exact repayment is final on Base by `D`, but Leo never claims before `C`, because he lost access to his Vault, is unavailable, or withholds `Y`. Missing `Y` alone is not the trigger and lender silence is not borrower default; at `C` the contract permanently disables the claim and freezes Ana's refund recipient, and anyone may then trigger a fixed-effect refund of the exact debt to that recipient. Before `C`, Ana's committed protocol account may irreversibly switch that recipient to itself. Delayed automation does not expire the refund right, and once the refund is final Leo can never also collect the USDC. The refund does not depend on the ECVS, which must never gate it, and it does not by itself unlock the bitcoin.

The bitcoin, however, stays in escrow, because Bitcoin cannot see the Base refund. What can happen to it next depends on records that disagree in part:

- The conditional ECVS route exists only for a loan created after its production-activation gate, with the secret's hash committed before the escrow was built. For such a loan the ECVS verifies the exact finalized refund event, publishes `S`, and the platform inserts `S` into the same borrower-signed recovery transaction; publication or broadcast alone is not recovery. Today this route is unavailable: counsel approval and implementation evidence have not activated it.
- Leo's lender-recovery package remains Bitcoin-valid after `T`, 24 hours after `C`, and pays his fixed destination. No hacker is required for the borrower to lose collateral after paying: Leo, or someone holding a copy of his signed package, can broadcast it; the holder cannot redirect the fixed payout, and the application cannot revoke a valid package held outside it.
- Both participants may agree to a cooperative settlement, as long as no other terminal spend has confirmed.
- The borrower long-stop is an open question. The long-stop constraint requires every newly constructed escrow to include borrower long-stop candidates, and the default-timing invariant makes such a package valid after `B = D + 864,000` seconds (ten days) without `Y` or `S`, racing the still-valid lender route until one confirms; the long-stop recovery flow owns that route. The security record on this case says neither the long-stop nor the `S` route is protection until implemented for the escrow and confirmed before the lender transaction, and whether the long-stop variants are among the signed terminal packages also turns on the open terminal-package count. The missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route, the residual-risk invariant requires documenting an unsupported borrower-Bitcoin outcome, and the unclaimed-repayment flow lists the remaining routes without the long-stop. Whether these describe different loans or contradict each other is undecided.

In every case Bitcoin settles on the one spend that confirms first, and the product may not claim that Ana's collateral is currently recoverable.

When the borrower does not repay

If no exact repayment is final by D, the loan is classified as ordinarily defaulted for product routing. Missing Y, watcher silence, a raw transfer or an unavailable fallback is never substituted for missing repayment, and an observer conflict pauses the classification instead of inferring default. Leo's route still waits for Bitcoin consensus time to pass T. In this version the organization never schedules or broadcasts a default transaction: once the timelock condition is valid and after fresh reauthentication, Leo may export his current signed package and broadcast it himself. Anyone holding a copy can broadcast it too, but the payout goes only to Leo's fixed destination, and a copied package cannot be recalled by application status. It is final only when it confirms. Automatic organization broadcast is deferred. A cooperative settlement remains available until another terminal spend confirms.

When both participants want to settle differently

Borrower and lender may agree on a different split of the unspent escrow. Each freshly signs the same six agreed variants with their existing loan key, and the platform may broadcast one variant once both signatures are stored and the archive receipt validates; one-sided consent, a lost vault password or any mismatch stops the attempt and leaves earlier routes intact. The Bitcoin mechanism is defined, but the signing-session experience, the matching Base accounting and support procedures are deferred, so the product does not offer this as a complete operational capability.

When observers disagree or are unavailable

If Base observation sources conflict or are both unavailable, the system keeps its finalized projections and blocks only new Base-derived consequential actions until replay recovers; a reorganization correction rolls the current view back to the last common finalized block and replays history. If Bitcoin observers disagree or are unavailable, packages and authority are preserved, an alert is raised, and only the automation that depends on the contested fact stops, with no manual override; it resumes only on fresh agreement.

When Cloudflare is down

All production application artifacts in this version run on Cloudflare: the product, the application interface with its D1 database, the Base observer, the Vault and the Bitcoin operations container. They keep separate builds, credentials, origins and releases, but they share one provider failure domain. A Cloudflare-wide outage can therefore suspend the product, the Vault, the application interface and database, Base observation, Bitcoin automation and the operator surface together. The chains continue, but the application accepts no new consequential command and claims no chain result. After Cloudflare returns, the system replays and reconciles Base and Bitcoin before resuming its pending jobs.

There is no live fallback on another provider, no active-active deployment, no guaranteed cold redeployment and no stated recovery time or recovery point objective. The encrypted backups outside Cloudflare protect the signed package data, so a provider or company disappearance need not force fresh signatures after a successful backup, but they are not a running service, and

the newest package set not yet backed up remains exposed to loss. An outage also has a cost beyond inconvenience: after a secret has become public, it can delay a required Bitcoin action and worsen a refund or terminal-route race.

When a component or credential is compromised

No threat is summarized by a severity word; the analysis states the compromised capability, the exact lifecycle state, the direct consequence and the recovery opportunity. Combinations deserve more attention than isolated leaks, because independently insufficient capabilities can collapse into one shared failure domain or become useful at a later state:

- A thief of the signed package archive cannot change any committed destination or amount, but once the matching public secret or matured timelock exists can force that fixed route, including a default after valid repayment.
- A malicious shared frontend that captures both participants' keys at different moments is the dominant common-mode threat: once escrow exists, it can steal the collateral through the cooperative route, and it is more important than two unrelated account takeovers.
- A compromise of Base upgrade or administration authority could redirect or lock many users' USDC if upgrades could alter existing funded loans. Who controls upgrades in this version is undecided: some records keep a 2-of-3 Safe through a seven-day timelock, while the upgrade-governance selection gives a single key control during a short public mainnet beta (see Open questions).
- Product account takeover alone is not Vault decryption; it exposes metadata and supplies the ciphertext half of a possible offline password attack.

What the system cannot guarantee

- It cannot make Base acceptance and Bitcoin activation atomic, so the lender bears the activation-pending risk described above.
- It cannot stop a lender, or a holder of the lender's signed package, from taking the collateral after the borrower repaid but the lender stayed silent, unless a borrower route confirms first; a holder can only send it to the lender's fixed destination, never redirect it; whether such a loan has a borrower long-stop route is undecided, since a build constraint requires every new escrow to include one and the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route.
- It cannot recover a forgotten vault password. Product account recovery, support, ciphertext restore or company backup does not recover the vault's plaintext seed or secrets.
- It offers no live continuity during a Cloudflare-wide outage.
- It has no liquidation: selling bitcoin, returning surplus and enforcing shortfalls are outside this version, and so are public or multi-lender matching, pooled capital, product custody and automatic capital deployment.

What has been built and what is claimed

The required outcome of this design is a developer-operated local demonstration: two isolated actors use the real product, application interface, Vault, Base contracts, observer, Bitcoin operations and protocol library through their public seams to complete one representative loan through normal repayment and confirmed borrower recovery. It follows the same journey described above; there is no demo-only lifecycle. Demo identities, tokens, wallets, balances, contracts and provider substitutes exist only inside that local profile, and any public credential, non-local chain or non-loopback setting makes it fail closed before provisioning.

That demonstration is the target, not a current capability. The hand-off sequence records the local-demo runtime as still in progress: its supervisor still contains an explicit not-implemented stop and its verification steps have not passed. The design also targets deterministic local verification, a separate claim that would prove neither a developer-runnable product nor a deployment; its verification records are planned and unverified. It makes no testnet or mainnet deployment claim and no activated-operation claim: those need their own later credentials, funding, deployment, monitoring, legal and release gates, and an intentional enablement action. A configuration name or a successful local run activates nothing.

Choices and their reasons

- **Five domains.** The domains separate agreement meaning, participant Bitcoin authority, Base value movement, verified observation with the conditional fallback, and durable application coordination, so that progress in one never stands in for another's authority. The user approved adding Participant Access And Loan Operations as the product application layer separate from the protocol domains; the other four domains were chosen by the agent.
- **Base L1 finality before any consequential hand-off.** One uniform rule across funding, secret disclosure, repayment and refund, at the cost of roughly 20 minutes of waiting per hand-off, prevents a transient Base observation from triggering a cross-chain action.
- **One cloud provider.** Keeping every production artifact on Cloudflare, with separate accounts, projects and credentials where they limit blast radius, is an explicit simplicity trade-off accepted instead of multiplying failure domains before those operations are proven. It is not a continuity claim.
- **Pre-signed fixed-destination packages with fixed fee variants.** Participants authorize exact transactions in advance, so later execution needs only a public condition and a holder of a stored package cannot redirect value; fees are fixed because the transaction is signed before use.
- **Manual lender-default broadcast.** The security analysis records that leaving default broadcast to the lender reduces the organization's execution authority but gives the lender a direct ability to bypass application policy after maturity.
- **An ECVS without a Bitcoin key.** The selected fallback design gives the service no Bitcoin key and no discretionary judgment; its secret can still be disclosed too early, withheld, lost or compromised, which is why independent controls and the production-activation gate remain material. Only a loan created after that gate, with the hash of `S` committed in its terms, can use it.

Open questions

The records leave these points undecided. Each side is stated as its record holds it; none is settled until the user answers.

- **Approval of final terms.** The architecture record says publication is standing acceptance and reservation creates the canonical loan with no second lender approval, and the offer-and-reservation interface, the multi-lender exclusion record and the offer selection also say there is no second lender approval; the loan-terms responsibility, the terms interface and the origination flow say both roles explicitly approve the same candidate and the second matching approval commits it. Neither is settled.
- **Fee when no signed tier qualifies.** The fee interface's payload says to select the 200 satoshis-per-virtual-byte tier, record CPFP eligibility and alert for a separate manual fee approval, and two ticket resolutions likewise use the 200 tier with CPFP eligibility and an alert; the same interface's tier-selection action says to fail closed and select nothing. Neither is settled.
- **Number of terminal packages.** One sentence of the package-construction responsibility and the durable package-archive constraint speak of a twelve-package terminal phase, while the package-construction responsibility's own construction statement, the manifest interface and the Bitcoin state records specify six borrower-recovery, six lender-recovery and six borrower long-stop variants, eighteen in all. Neither count is settled.
- **Offer identity in the terms value.** The architecture record and the loan-terms responsibility say `LoanTermsV1` carries the selected offer revision and reservation number; the terms interface says that identity belongs to the offer-and-reservation contract, not the terms value. The records do not declare this disagreement, and neither side is settled.
- **Base upgrade control.** The upgrade invariant, the immutable-conversion scope record and the settlement responsibility say this version keeps upgrade administration under a 2-of-3 Safe through a seven-day timelock; the upgrade-governance selection says a single upgrade key controls local, test and a short public mainnet beta before ownership moves to that timelock. Neither is settled.
- **On-chain pause.** The settlement responsibility, the upgrade selection and the window-opening constraint say there is no on-chain pause; the deployment and upgrade state record lists pause roles and a paused state. Whether that state is an application-level stop or an on-chain role is undecided.
- **Borrower long-stop after valid repayment.** As described under When the lender stays silent after valid repayment, the long-stop constraint requires a long-stop route at `B` in every new escrow, while the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route. This is undecided.
- **Vault payment account status.** As described under What binds everything to one loan, the records disagree whether the payment capability is only selected and gated or already implemented. This is undecided; the guide treats it at most as a selected design.

The technical architecture record holds the exact purpose, binding, authority and boundaries, and each domain guide explains its domain in full.

Other guides

This guide refers to these separate documents of the System Guide:

- Loan Terms And Commitments (domain guide)
- Bitcoin Authorization And Recovery (domain guide)
- Base USDC Settlement And Funding (domain guide)
- Base Observation And Fallback Review (domain guide)
- Participant Access And Loan Operations (domain guide)

Technical records

The exact rules, contracts and conditions live in these records of the design package:

- **Architecture record:** Bitcoin-Backed USDC One-Loan System Architecture.